introduce ambiguity. `?int|string` could read as if only `int` was nullable. "But if the int is nullable then, logically, so is the string," you might argue, and you'd be entirely correct. But figuring that out is additional cognitive load, and our jobs are already difficult enough, so nullable union types are only achieved by adding `|null` to the declaration.

Intersection Types

The corollary to the union type is the intersection. With an intersection type declaration, we require that the data in question matches *all* the set of types (rather than just one). Logically, this restricts the valid members of an intersection to classes, interfaces, and enums because it's impossible for a data type to be both, say, a string and an integer at the same time. Even if PHP code often reads as if data somehow has multiple types, remember that internally the data type is being cast as required and can only be of one type at any particular time.

Imagine we have a function that needs to both iterate and count data. If we supply an object, it would be good to know it could be counted (e.g., with `count()`) and accessed like an array (e.g., with `foreach`). PHP 8.1 introduces the "intersection type," which is a special composite type, which defines a set of classes or interfaces, all of which the incoming (or outgoing) data must match. If union types are a logical OR, then intersections are a logical AND.

The code below illustrates this but makes use of concepts we haven't covered yet. For now, just accept that `ArrayAccess` and `Countable` are the names of two built-in interfaces in PHP, and interfaces can be explicitly implemented by classes. The type declaration `ArrayAccess&Countable $data` means "`$data` is an object that implements these two interfaces." `ArrayObject` is a built-in class that meets that requirement.

```
declare(strict_types=1);

function countAndIterate(ArrayAccess&Countable $data)
{
    print "Data contains " . count($data) . " things:" . PHP_EOL;

    foreach ($data as $datum) {
        var_dump($datum);
    }
}
```